Plan: viz + retrieval de-duplication (then the GUI demo)

HyMeKo — code-health consolidation

2026-08-12

1 Scope and goal

Remove two verified duplications surfaced by the audit (`reports/2026-08-12-code-duplication-audit.md`), each behind a parity gate; then continue the GUI demo on the consolidated helpers.

(A) Retrieval. The library `RetrievalDeliveryPolicy.predict` is reimplemented because it does not return the *source index* callers need for display/audit. Add `predict_with_source(x) -> (theta, nearest_idx)` and migrate the one *true* θ-dup (`video_r11_6c_composition._nearest`; `delivery_viewer` already migrated). Measured parity: $\max|\Delta\theta| = 0$.

(B) Visualization. `_cam()` ($\times 8$, ~ 3 presets) and `_Filmer` ($\times 3$, nearly identical) and ad-hoc `mujoco.Renderer` loops. Create one `viz/rollout_film.py` — `class Filmer(cam, phase=None)` (the frame_hook), `CAMS` presets (`top_down` 1.02/−87/90, `angled` 0.9/−55/90, `side` 0.60/90/−68), and `render_qpos_seq(model, qseq, cam)` — and migrate `video_r11_6c_composition` (the template others import from) + `delivery_viewer.render_gif`.

NOT consolidated (documented, different algorithms). `coin_zero_home_rrt._nearest` (RRT tree nearest-neighbour), `r11_7b_physics_selector` (a physics-feature metric, not descriptor retrieval), `r11_6d_*` audits (return an id/distance metric, not θ). A grep matches them; they are not `predict` dups. Leaving them is the correct call.

2 Affected files

- `coin_delivery/delivery_bc/retrieval.py` — add `predict_with_source` (additive). + test.
- **NEW** `viz/rollout_film.py` + `tests/test_rollout_film.py`.
- `experiments/video_r11_6c_composition.py` — `_nearest`→`predict_with_source`; `_Filmer`/`_cam`→`viz.` (keep `r11_7b_flagship`'s imports working).
- `gui/delivery_viewer.py` — `render_gif`→`render_qpos_seq`.

3 CORE.YAML items touched

None (all `hymeko_rl/**`, non-core). No new dependency.

4 Interface changes

Additive only: `RetrievalDeliveryPolicy.predict_with_source`; new `viz.rollout_film` symbols. `video_r11_6c_composition` keeps exporting `_Filmer`, `_cam`, `_trim_settled` (as re-exports from `viz.rollout_film`) so `r11_7b_flagship_video`'s imports do not break.

5 Test strategy

- Unit: `predict_with_source` returns `(predict(x), argmin_idx)` — θ equals `predict`, index is the nearest row (leave-one-out honoured).
- Unit: `Filmer` captures N frames of shape $(H, W, 3)$ + dtz; `CAMS` presets build valid cameras; `render_qpos_seq` renders each state.
- **Parity gate (retrieval)**: over ≥ 8 scenarios, migrated `video_r11_6c` θ equals the pre-migration θ ($\Delta = 0$).
- **Parity gate (viz)**: render `video_r11_6c_composition` one delivery clip before and after; the frame arrays must be **byte-identical** (`render_state_signature` / array equality). Deterministic renders, so $\Delta = 0$ or halt.
- Static: `ruff`; `mypy --strict` on `retrieval.py` + `rollout_film.py`.

6 Performance budget & risk

Peak RSS < 2 GB. The viz parity gate re-renders one clip twice ($\sim$1–2 min each) — background. Risks: (i) the generalized `Filmer` changes a frame (camera/renderer order) — caught by the byte-parity gate, halt if non-zero; (ii) `predict_with_source` index off-by-one vs the manual argmin — caught by the θ-parity + the id equality; (iii) breaking `r11_7b_flagship`'s imports — mitigated by re-exporting the moved symbols.

7 Rollback

Each consolidation is its own commit behind its parity gate; revert restores the local `_nearest`/`_cam`/`_Filmer`. No state mutation.

8 Then: GUI demo

On the consolidated helpers, finish the delivery demo (shape/target/strategy over the real pipeline). Separate step, after both parity gates are green.